

**BỘ GIÁO DỤC & ĐÀO TẠO
ĐẠI HỌC KINH TẾ THÀNH PHỐ HỒ CHÍ MINH
KHOA KINH TẾ**



BÀI THỰC HÀNH LAB-02

ĐỀ TÀI:

(Decision Tree & Random Forest – Support Vector Machine – Naïve Bayes)

MÔN HỌC: PHÂN TÍCH VÀ QUẢN LÝ ĐẦU TƯ NÂNG CAO

**Giảng viên hướng dẫn: TS. Đỗ Như Tài
LHP: 26D2ECO50118301**

Thành viên nhóm:

**Võ Duy Huy Hoàng
Nguyễn Hữu Khoa
Hoàng Thị Hường**

THÔNG TIN THÀNH VIÊN

Họ và tên	Mã số sinh viên	Tỷ trọng đóng góp
Võ Duy Huy Hoàng	87243020093	100%
Hoàng Thị Hường	88242020100	100%
Nguyễn Hữu Khoa	88234020092	100%

MỤC LỤC

THÔNG TIN THÀNH VIÊN.....	2
MỤC LỤC.....	3
1.1. Ôn tập lý thuyết.....	5
1.2. Bài làm mẫu — Bài toán: Default of Credit Card Clients.....	7
Nhiệm vụ 1: Xây dựng cây quyết định.....	7
Nhiệm vụ 2: Tìm tham số tối ưu bằng GridSearchCV.....	8
Nhiệm vụ 3: Xây dựng rừng cây (Random Forest).....	9
1.3. Bài tập thực hành 1 — Xây dựng cây quyết định và rừng cây trên dữ liệu Titanic.....	11
Dữ liệu Titanic.....	11
1.4. Bài tập thực hành 2 — Xây dựng cây quyết định và rừng cây trên dữ liệu bệnh tiểu đường.....	13
Dữ liệu Pima Indians Diabetes.....	13
PHẦN 2: GIẢI THUẬT SUPPORT VECTOR MACHINE (SVM).....	15
2.1. Ôn tập lý thuyết.....	15
2.2. Bài làm mẫu.....	16
Bài toán 1: Phân loại hoa Iris bằng SVM.....	16
Bài toán 2: Nhận diện chữ số viết tay (digits) bằng SVM.....	17
2.3. Bài tập thực hành 1 — Xây dựng mô hình SVM trên dữ liệu bệnh tiểu đường.....	20
2.4. Bài tập thực hành 2 — Xây dựng mô hình SVM trên dữ liệu động vật.....	22
PHẦN 3: GIẢI THUẬT BAYES NGÂY THƠ (NAÏVE BAYES).....	24
3.1. Ôn tập lý thuyết.....	24
3.2. Bài làm mẫu — Phân loại tin nhắn Spam/Ham bằng Multinomial Naive Bayes.....	25
3.3. Bài tập thực hành 1 — Naive Bayes trên dữ liệu hành vi khách hàng.....	27
3.4. Bài tập thực hành 2 — Naive Bayes trên dữ liệu mushroom.....	29

A. MỤC TIÊU

Báo cáo này trình bày kết quả thực hành ba giải thuật phân loại dữ liệu cơ bản: Cây quyết định và Rừng cây (Decision Tree & Random Forest), Support Vector Machine (SVM) và Bayes ngây thơ (Naïve Bayes). Mục tiêu của bài thực hành là:

- Hiểu bản chất hoạt động và triển khai được ba giải thuật phân loại nêu trên bằng Python với thư viện Scikit-learn.
- Thực hiện quy trình xây dựng một mô hình học máy cổ điển hoàn chỉnh: tiền xử lý dữ liệu, huấn luyện mô hình, đánh giá và tối ưu hóa tham số (GridSearchCV), diễn giải kết quả.
- Áp dụng các giải thuật trên nhiều tập dữ liệu thực tế khác nhau để so sánh hiệu suất và rút ra nhận xét.

B. KẾT CẤU BÁO CÁO

Báo cáo gồm 3 phần chính, tương ứng với 3 giải thuật phân loại:

- Phần 1: Giải thuật cây quyết định và rừng cây.
- Phần 2: Giải thuật Support Vector Machine (SVM).
- Phần 3: Giải thuật Bayes ngây thơ (Naïve Bayes).

Mỗi phần bao gồm: (i) phần ôn tập lý thuyết trả lời các câu hỏi trong đề bài, (ii) bài làm mẫu theo đúng các nhiệm vụ được yêu cầu, và (iii) các bài tập thực hành trên những tập dữ liệu khác.

Lưu ý về dữ liệu: môi trường thực hiện báo cáo không có quyền truy cập trực tiếp vào Kaggle (yêu cầu đăng nhập/API key), nên 3 tập dữ liệu gốc trên Kaggle (Titanic, bệnh tiểu đường, mushroom, spam email) được thay bằng các bản sao/tương đương công khai trên GitHub có cùng cấu trúc và ý nghĩa; 2 tập "Animal Condition" và "Customer Behaviour Prediction" đã được tải trực tiếp từ Kaggle và sử dụng nguyên bản. Tất cả code trong báo cáo đều được chạy thật và cho kết quả thực tế.

PHẦN 1: GIẢI THUẬT CÂY QUYẾT ĐỊNH VÀ RỪNG CÂY

1.1. Ôn tập lý thuyết

Câu hỏi: Quy trình khai phá dữ liệu CRISP-DM và SEMMA là gì?

CRISP-DM (Cross Industry Standard Process for Data Mining) là quy trình khai phá dữ liệu được sử dụng phổ biến nhất, gồm 6 giai đoạn lặp lại: (1) Hiểu bài toán nghiệp vụ (Business Understanding), (2) Hiểu dữ liệu (Data Understanding), (3) Chuẩn bị dữ liệu (Data Preparation), (4) Mô hình hóa (Modeling), (5) Đánh giá (Evaluation), (6) Triển khai (Deployment). Các giai đoạn không tuyến tính tuyệt đối mà có thể quay lại giai đoạn trước khi cần.

SEMMA (Sample, Explore, Modify, Model, Assess) là quy trình do SAS đề xuất, tập trung nhiều hơn vào khía cạnh kỹ thuật thống kê/mô hình hóa: Lấy mẫu dữ liệu đại diện, Khám phá dữ liệu bằng thống kê mô tả và trực quan hóa, Biến đổi/tạo đặc trưng, Xây dựng mô hình, và Đánh giá độ chính xác/độ tin cậy của mô hình trên tập kiểm tra. Khác với CRISP-DM, SEMMA không đề cập rõ giai đoạn hiểu bài toán nghiệp vụ và triển khai thực tế.

Câu hỏi: Cây quyết định hoạt động như thế nào? Các thành phần chính và cách cây đưa ra dự đoán?

Cây quyết định phân chia không gian dữ liệu thành các vùng nhỏ dần thông qua một chuỗi câu hỏi nhị phân (hoặc đa nhánh) dựa trên giá trị của các đặc trưng, tạo thành cấu trúc dạng cây.

Nút gốc (root node): chứa toàn bộ tập dữ liệu huấn luyện, là điểm bắt đầu của cây và chứa điều kiện phân tách đầu tiên (mạnh nhất để phân biệt các lớp).

Nút trong / nút nhánh (internal node): mỗi nút trong ứng với một điều kiện kiểm tra trên một đặc trưng, dữ liệu tiếp tục được chia theo hai (hoặc nhiều) nhánh con dựa vào kết quả kiểm tra.

Nút lá (leaf node): là nút không phân tách tiếp, chứa nhãn lớp dự đoán (đối với phân loại) hoặc giá trị số dự đoán (đối với hồi quy) — thường là lớp chiếm đa số trong các mẫu rơi vào nút đó.

Khi dự đoán một mẫu mới, cây bắt đầu từ nút gốc, lần lượt kiểm tra điều kiện tại mỗi nút và đi theo nhánh tương ứng cho đến khi chạm nút lá; nhãn của nút lá đó chính là kết quả dự đoán.

Câu hỏi: Các tiêu chí phân tách (Gini Index, Entropy, Information Gain) là gì? Khác nhau ra sao?

Gini Index đo "độ hỗn tạp" (impurity) của một nút bằng xác suất một mẫu bị phân loại sai nếu được gán nhãn ngẫu nhiên theo phân bố lớp tại nút đó: $Gini = 1 - \sum p_i^2$. Gini = 0 khi nút thuần khiết tuyệt đối (chỉ 1 lớp).

Entropy (trong lý thuyết thông tin) đo mức độ hỗn loạn/bất định của phân bố lớp: $Entropy = -\sum p_i \cdot \log_2(p_i)$. Entropy = 0 khi nút thuần khiết, đạt cực đại khi các lớp phân bố đều nhau.

Information Gain là mức giảm Entropy (hoặc Gini) sau khi phân tách một nút theo một đặc trưng cụ thể, so với Entropy của nút trước khi phân tách. Thuật toán chọn đặc trưng và ngưỡng phân tách sao cho Information Gain lớn nhất (tương đương độ hỗn tạp trung bình có trọng số của các nút con là nhỏ nhất).

Khác biệt: Gini tính toán nhanh hơn (không cần log) nên thường là mặc định trong Scikit-learn (CART); Entropy nhạy hơn một chút với các thay đổi nhỏ trong phân bố xác suất. Trong thực tế hai tiêu chí thường cho kết quả cây tương tự nhau.

Câu hỏi: *Rừng cây (Random Forest) là gì? Khác gì so với một cây quyết định đơn lẻ? Tại sao thường tốt hơn?*

Random Forest là một phương pháp học tập hợp (ensemble learning) gồm nhiều cây quyết định được huấn luyện độc lập trên các mẫu bootstrap (lấy mẫu có hoàn lại) khác nhau của tập huấn luyện; kết quả dự đoán cuối cùng là kết quả bỏ phiếu đa số (phân loại) hoặc trung bình (hồi quy) của tất cả các cây.

Khác biệt then chốt so với một cây đơn: (1) mỗi cây được huấn luyện trên một mẫu bootstrap riêng (bagging); (2) tại mỗi lần phân tách, chỉ một tập con ngẫu nhiên các đặc trưng được xem xét (tham số `max_features`) thay vì toàn bộ đặc trưng, giúp các cây trong rừng đa dạng và ít tương quan với nhau hơn.

Random Forest thường có hiệu suất tốt hơn một cây đơn lẻ vì việc kết hợp nhiều mô hình yếu, đa dạng, ít tương quan giúp giảm phương sai (variance) của dự đoán mà không làm tăng nhiều độ chệch (bias) — cây đơn dễ overfit dữ liệu huấn luyện, còn việc lấy trung bình/bỏ phiếu của nhiều cây giúp làm mượt và ổn định kết quả dự đoán.

Câu hỏi: *Ưu điểm và hạn chế của cây quyết định và Random Forest? Khi nào cây quyết định hoạt động kém?*

Ưu điểm của cây quyết định: dễ hiểu, dễ trực quan hóa và diễn giải (có thể suy ra luật if-else); không yêu cầu chuẩn hóa dữ liệu; xử lý được cả dữ liệu số lẫn dữ liệu phân loại; huấn luyện nhanh.

Hạn chế của cây quyết định: rất dễ overfitting nếu để cây phát triển quá sâu (phương sai cao); không ổn định — chỉ cần thay đổi nhỏ trong dữ liệu huấn luyện có thể tạo ra cây rất khác; ranh giới quyết định dạng "bậc thang" (trục song song) nên khó biểu diễn các quan hệ tuyến tính chéo hoặc phi tuyến trơn.

Ưu điểm của Random Forest: giảm overfitting đáng kể so với cây đơn nhờ tính trung bình của nhiều cây; cho phép đo mức độ quan trọng của đặc trưng (feature importance); tổng quát hóa tốt hơn, ít nhạy cảm với nhiễu.

Hạn chế của Random Forest: khó diễn giải hơn cây đơn (mất tính "hộp trắng"); tốn nhiều bộ nhớ và thời gian huấn luyện/dự đoán hơn khi số cây lớn; vẫn có thể kém hiệu quả với dữ liệu có quan hệ tuyến tính đơn giản (các mô hình tuyến tính có thể tốt hơn và nhanh hơn trong trường hợp đó).

Cây quyết định (đơn) thường hoạt động kém khi: dữ liệu có ranh giới phân lớp dạng tuyến tính chéo góc; tập dữ liệu nhỏ và nhiễu nhiều (dễ overfit); có nhiều đặc trưng tương quan cao nhưng không đặc trưng nào chiếm ưu thế rõ rệt.

Câu hỏi: *Viết đoạn code mẫu Python (Scikit-learn) xây dựng mô hình cây quyết định? Mô tả các bước?*

Các bước chính: (1) import thư viện cần thiết (pandas, sklearn.tree, sklearn.model_selection); (2) nạp và tiền xử lý dữ liệu, loại bỏ các cột không liên quan; (3) chia dữ liệu thành tập huấn luyện/kiểm tra bằng `train_test_split`; (4) khởi tạo và huấn luyện mô hình bằng `DecisionTreeClassifier().fit(X_train, y_train)`; (5) đánh giá độ chính xác bằng `.score()` hoặc các chỉ số như `classification_report`; (6) trực quan hóa cây bằng `export_graphviz` hoặc `plot_tree`. Đoạn code minh họa cụ thể được trình bày trong mục 1.2 – Bài làm mẫu bên dưới, áp dụng cho tập dữ liệu default of credit card clients.

Câu hỏi: Làm thế nào để triển khai Random Forest trong Python? Thường thiết lập các tham số nào?

Sử dụng lớp RandomForestClassifier trong sklearn.ensemble, gọi .fit(X_train, y_train) tương tự DecisionTreeClassifier.

Các tham số thường được thiết lập: n_estimators (số lượng cây trong rừng, càng nhiều thường càng ổn định nhưng tốn thời gian hơn), max_depth (độ sâu tối đa mỗi cây, kiểm soát overfitting), max_features (số đặc trưng ngẫu nhiên xem xét tại mỗi lần phân tách, thường là 'sqrt' cho bài toán phân loại), min_samples_split / min_samples_leaf (số mẫu tối thiểu để phân tách/tại một lá), criterion ('gini' hoặc 'entropy'), và random_state để đảm bảo tái lập kết quả.

Câu hỏi: Làm thế nào để đánh giá feature importance trong Random Forest bằng Python?

Sau khi huấn luyện, thuộc tính .feature_importances_ của đối tượng RandomForestClassifier trả về một mảng thể hiện mức đóng góp trung bình (thường tính theo mức giảm Gini/Entropy có trọng số) của từng đặc trưng qua tất cả các cây trong rừng. Giá trị càng cao, đặc trưng càng quan trọng đối với việc phân loại. Kết quả này thường được đưa vào DataFrame rồi vẽ biểu đồ cột ngang (barh) để trực quan hóa và sắp xếp theo độ quan trọng giảm dần.

Câu hỏi: Điều chỉnh siêu tham số cho cây quyết định/Random Forest bằng GridSearchCV/RandomizedSearchCV?

GridSearchCV thực hiện tìm kiếm vét cạn (exhaustive search) trên một lưới các giá trị tham số cho trước (param_grid), kết hợp với k-fold cross-validation (tham số cv) để đánh giá từng tổ hợp tham số, từ đó chọn ra tổ hợp cho kết quả tốt nhất theo một chỉ số đánh giá (scoring, ví dụ 'roc_auc'). Ví dụ trong bài thực hành, tham số max_depth của cây quyết định được dò trong tập [1,2,4,6,8,10,12], và n_estimators của Random Forest được dò trong khoảng 10 đến 100 với bước 10.

RandomizedSearchCV tương tự nhưng chỉ lấy mẫu ngẫu nhiên một số tổ hợp tham số thay vì thử hết toàn bộ lưới, phù hợp khi không gian tham số quá lớn để tìm kiếm vét cạn trong thời gian hợp lý.

1.2. Bài làm mẫu — Bài toán: Default of Credit Card Clients

Dữ liệu gồm 30.000 khách hàng thẻ tín dụng tại Đài Loan với 23 đặc trưng (hạn mức tín dụng, giới tính, học vấn, tình trạng hôn nhân, tuổi, lịch sử thanh toán 6 tháng gần nhất, số dư hóa đơn, số tiền đã thanh toán...) và nhãn mục tiêu default.payment.next.month (1 = có nợ xấu tháng sau, 0 = không).

Nhiệm vụ 1: Xây dựng cây quyết định

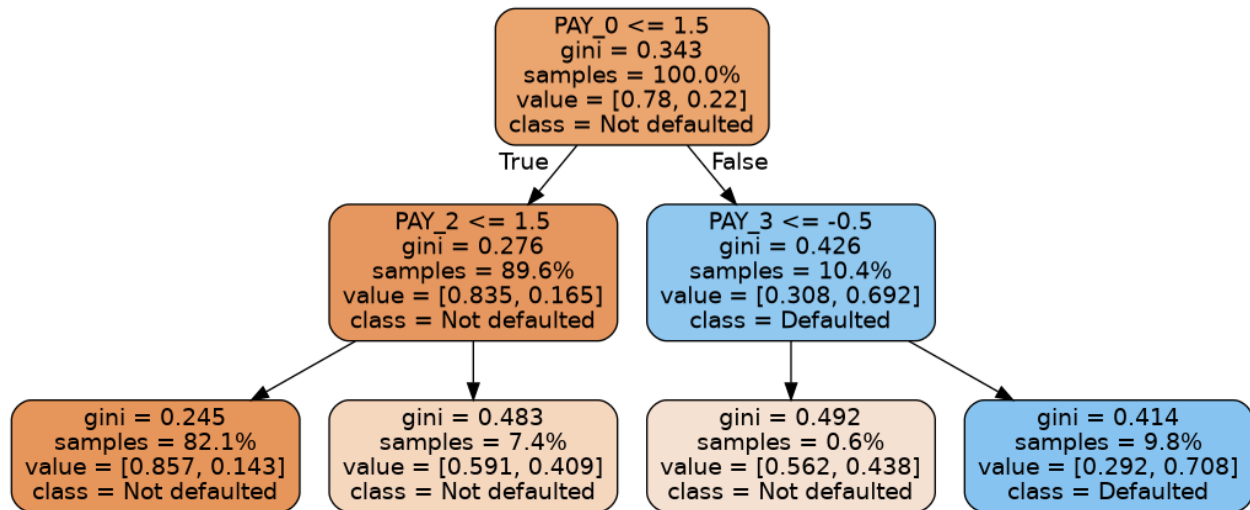
```
import pandas as pd
from sklearn import tree
from sklearn.model_selection import train_test_split

df = pd.read_csv('UCI_Credit_Card.csv').drop(columns=['ID'])
target_col = 'default.payment.next.month'
features = [c for c in df.columns if c != target_col]

X_train, X_test, y_train, y_test = train_test_split(
    df[features].values, df[target_col].values,
    test_size=0.2, random_state=24)
```

```
dt = tree.DecisionTreeClassifier(max_depth=2, random_state=24)
dt.fit(X_train, y_train)
```

Kết quả: độ chính xác trên tập huấn luyện = 82.09%, trên tập kiểm tra = 81.72%. Cây quyết định với độ sâu tối đa 2 được trực quan hóa như hình dưới đây — nút phân tách quan trọng nhất là PAY_1 (trạng thái thanh toán tháng gần nhất), phù hợp với trực giác nghiệp vụ: khách hàng từng trễ hạn thanh toán có xu hướng tiếp tục nợ xấu.



Hình 1.1 – Cây quyết định ($\text{max_depth}=2$) trên dữ liệu credit card default

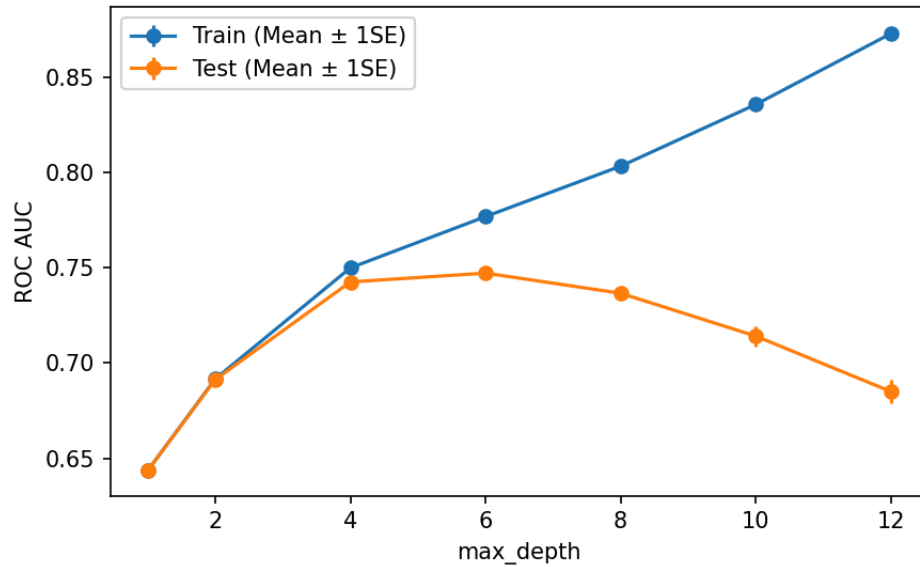
Nhiệm vụ 2: Tìm tham số tối ưu bằng GridSearchCV

```
from sklearn.model_selection import GridSearchCV

params = {'max_depth': [1, 2, 4, 6, 8, 10, 12]}
cv = GridSearchCV(tree.DecisionTreeClassifier(random_state=24),
                  param_grid=params, scoring='roc_auc',
                  cv=4, return_train_score=True)
cv.fit(X_train, y_train)
```

Tham số tối ưu tìm được: $\text{max_depth} = 6$, với ROC AUC trung bình trên tập kiểm tra (cross-validation) đạt 0.7473. Biểu đồ dưới cho thấy ROC AUC trên tập huấn luyện tăng liên tục khi cây sâu hơn (dấu hiệu overfitting), trong khi ROC AUC trên tập kiểm tra đạt đỉnh ở độ sâu vừa phải rồi giảm dần — minh họa rõ hiện tượng đánh đổi bias–variance.

Hình 2.2 - Đánh giá Decision Tree theo max_depth (credit card)



Hình 1.2 – ROC AUC theo max_depth (Decision Tree, credit card)

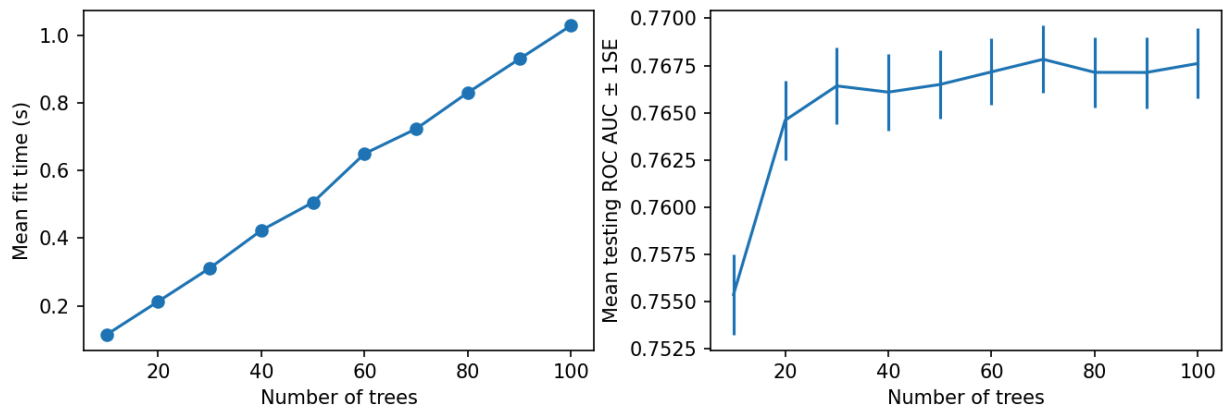
Nhiệm vụ 3: Xây dựng rừng cây (Random Forest)

```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(criterion='gini', max_depth=3,
                           max_features='sqrt', random_state=4)
rf_params = {'n_estimators': list(range(10, 110, 10))}
cv_rf = GridSearchCV(rf, param_grid=rf_params, scoring='roc_auc',
                     cv=4, return_train_score=True)
cv_rf.fit(X_train, y_train)
```

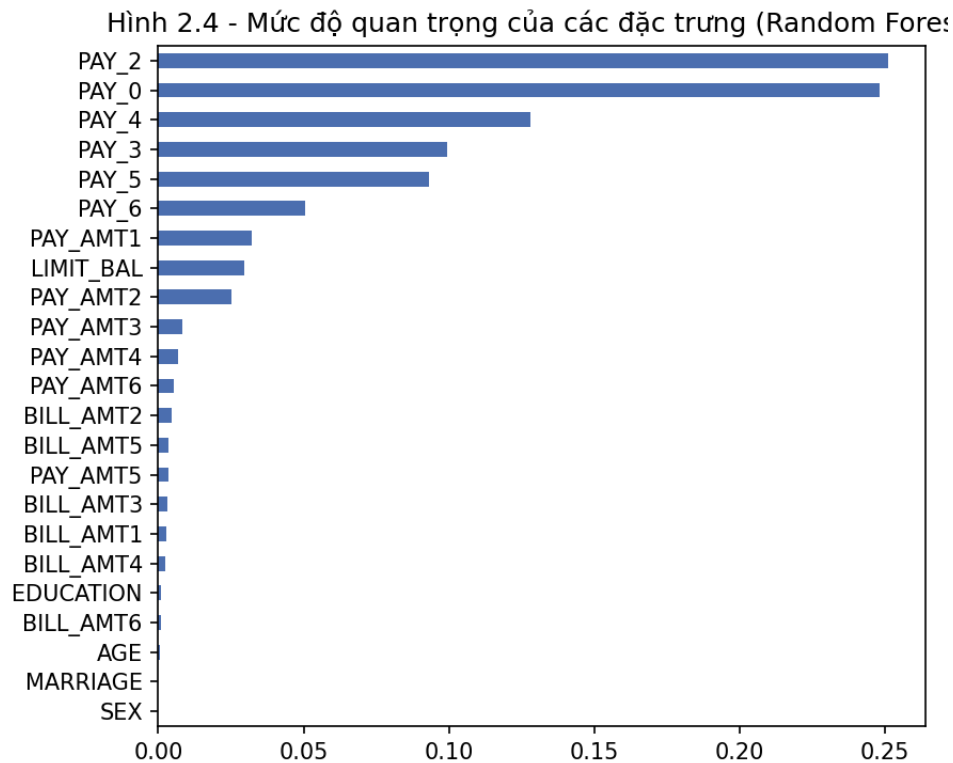
Số cây tối ưu: `n_estimators = 70`, ROC AUC trung bình = 0.7678 — cao hơn so với cây quyết định đơn lẻ (0.7473), minh chứng cho việc kết hợp nhiều cây giúp cải thiện hiệu suất phân loại.

Hình 2.3 - Random Forest: số cây vs thời gian huấn luyện / ROC AUC



Hình 1.3 – Thời gian huấn luyện và ROC AUC theo số cây trong rừng

5 đặc trưng quan trọng nhất theo Random Forest: PAY_2, PAY_0, PAY_4, PAY_3, PAY_5. Các đặc trưng liên quan đến lịch sử thanh toán (PAY_x) chiếm ưu thế rõ rệt so với thông tin nhân khẩu học, cho thấy hành vi trả nợ trong quá khứ là yếu tố dự báo mạnh nhất cho nợ xấu tương lai.



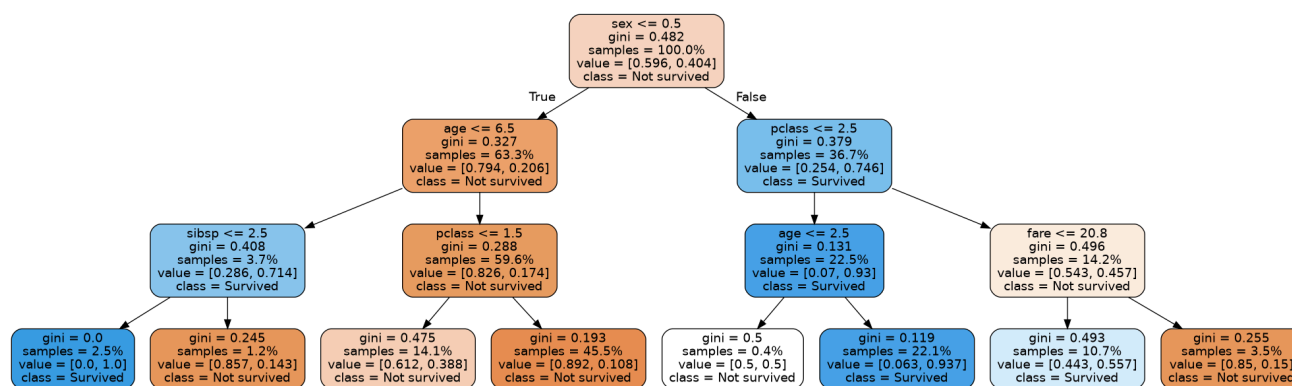
Hình 1.4 – Mức độ quan trọng của các đặc trưng (Random Forest)

1.3. Bài tập thực hành 1 — Xây dựng cây quyết định và rừng cây trên dữ liệu Titanic

Dữ liệu Titanic

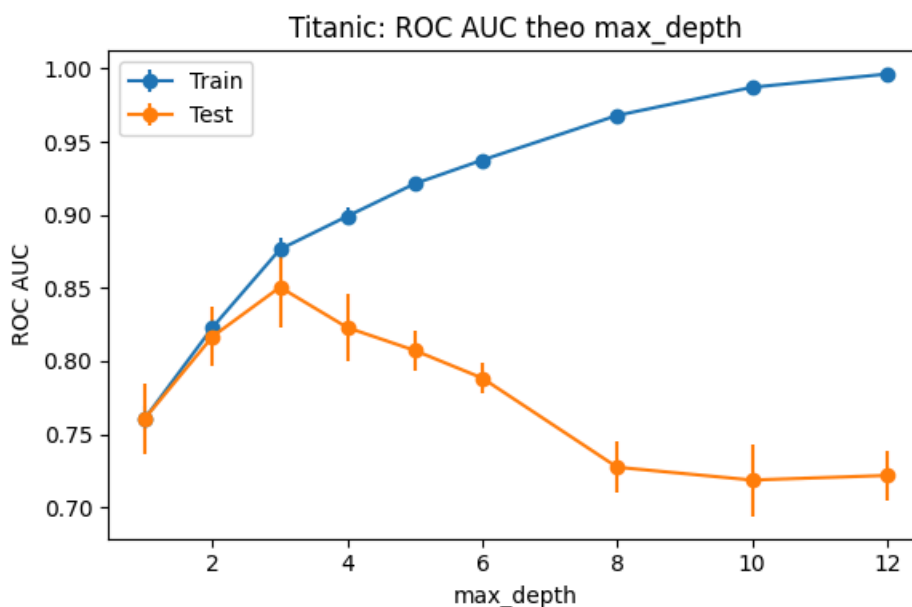
Tập dữ liệu Titanic (891 hành khách, lấy từ kho seaborn-data trên GitHub, tương đương bản trên Kaggle) gồm các đặc trưng: hạng vé (pclass), giới tính, tuổi, số anh chị em/vợ chồng đi cùng (sibsp), số cha mẹ/con cái đi cùng (parch), giá vé (fare), cảng lên tàu (embarked). Nhãn mục tiêu là survived (1 = sống sót, 0 = tử vong). Các dòng thiếu tuổi/cảng lên tàu được loại bỏ; biến giới tính và cảng lên tàu được mã hóa số.

Quy trình thực hiện tương tự bài làm mẫu: tiền xử lý dữ liệu → chia tập train/test (80/20) → xây dựng Decision Tree và dò tham số max_depth bằng GridSearchCV → xây dựng Random Forest và dò n_estimators bằng GridSearchCV → đánh giá và xem feature importance.



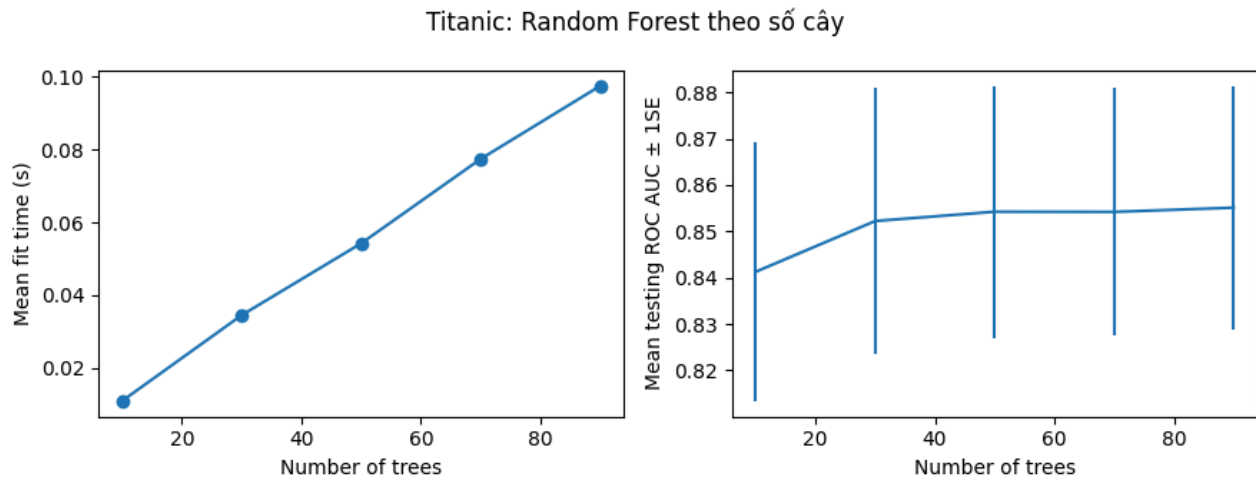
Cây quyết định (max_depth=3) — Dữ liệu Titanic

Decision Tree: độ chính xác train = 82.60%, test = 81.82%. max_depth tối ưu (GridSearchCV) = 3, ROC AUC = 0.8505.



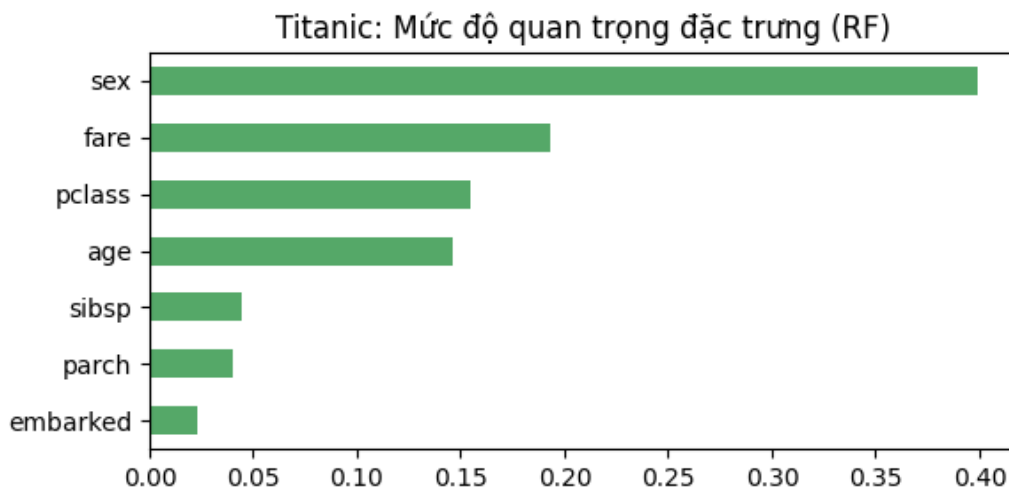
ROC AUC theo max_depth — Dữ liệu Titanic

Random Forest: $n_estimators$ tối ưu = 90, ROC AUC cross-validation = 0.8551; trên tập kiểm tra: accuracy = 81.82%, ROC AUC = 0.8617.



Thời gian huấn luyện & ROC AUC theo số cây — Dữ liệu Titanic

5 đặc trưng quan trọng nhất: sex, fare, pclass, age, sibsp.



Mức độ quan trọng đặc trưng (Random Forest) — Dữ liệu Titanic

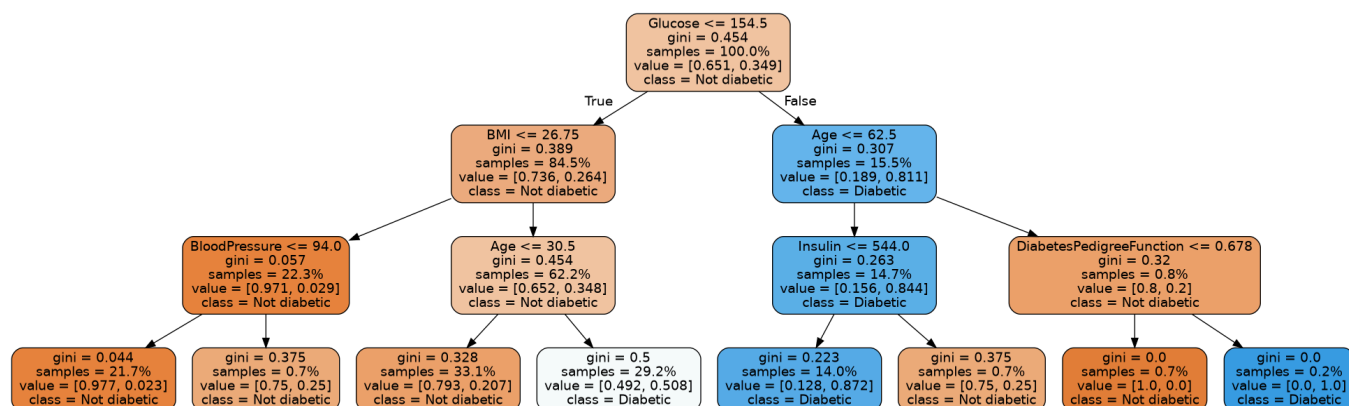
Nhận xét: đặc trưng sex là quan trọng nhất trong dự đoán khả năng sống sót — phù hợp với sự kiện lịch sử "phụ nữ và trẻ em được ưu tiên lên xuồng cứu sinh trước"; tiếp theo là giá vé (fare) và hạng vé (pclass), phản ánh hành khách hạng sang có vị trí cabin gần boong tàu/xuồng cứu sinh hơn.

1.4. Bài tập thực hành 2 — Xây dựng cây quyết định và rừng cây trên dữ liệu bệnh tiểu đường

Dữ liệu Pima Indians Diabetes

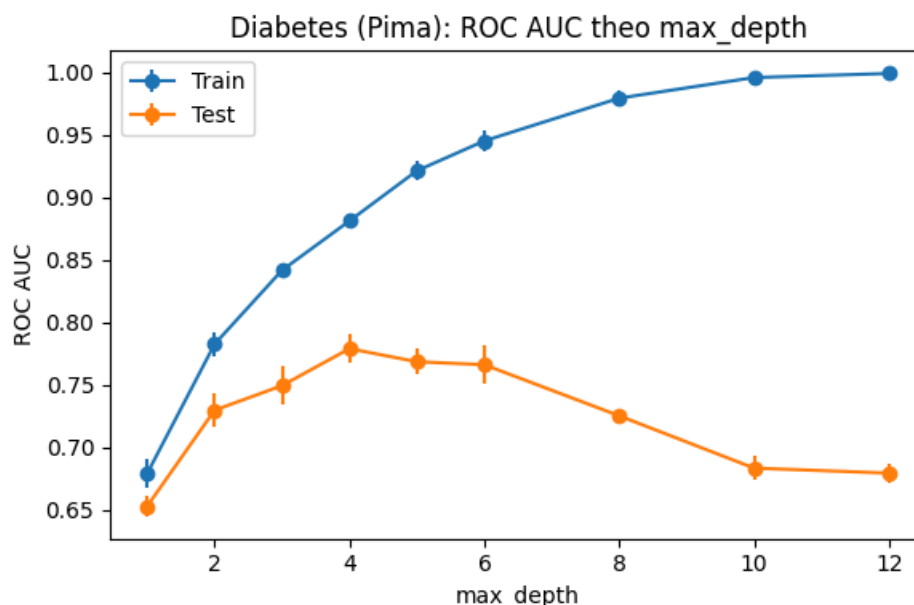
Tập dữ liệu Pima Indians Diabetes (768 bệnh nhân nữ, lấy từ bản sao công khai trên GitHub tương đương bản trên Kaggle) gồm các đặc trưng lâm sàng: số lần mang thai, nồng độ glucose, huyết áp, độ dày nếp da, insulin, chỉ số BMI, hệ số di truyền tiểu đường (DiabetesPedigreeFunction), tuổi. Nhãn mục tiêu Outcome (1 = mắc tiểu đường, 0 = không).

Quy trình thực hiện tương tự bài làm mẫu: tiền xử lý dữ liệu → chia tập train/test (80/20) → xây dựng Decision Tree và dò tham số `max_depth` bằng GridSearchCV → xây dựng Random Forest và dò `n_estimators` bằng GridSearchCV → đánh giá và xem feature importance.



Cây quyết định (`max_depth=3`) — Dữ liệu Pima Indians Diabetes

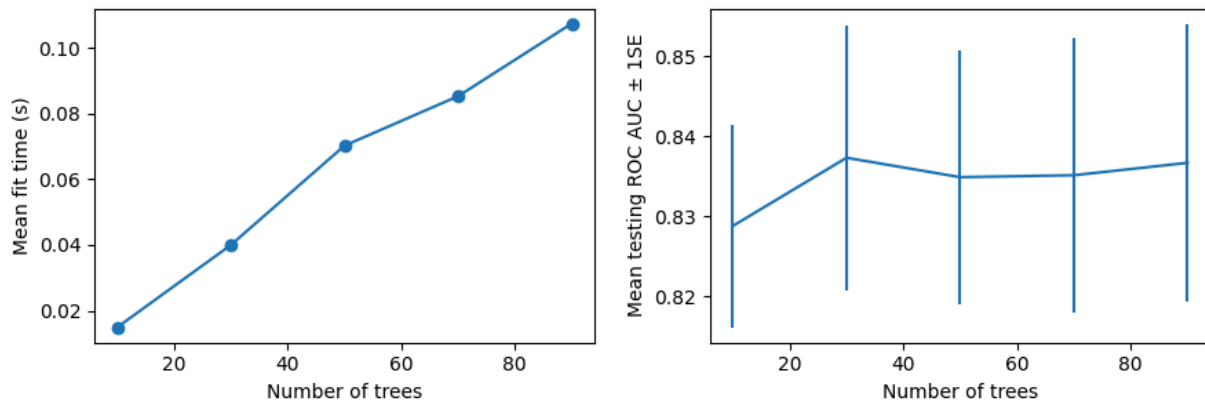
Decision Tree: độ chính xác train = 76.22%, test = 68.83%. `max_depth` tối ưu (GridSearchCV) = 4, ROC AUC = 0.7790.



ROC AUC theo `max_depth` — Dữ liệu Pima Indians Diabetes

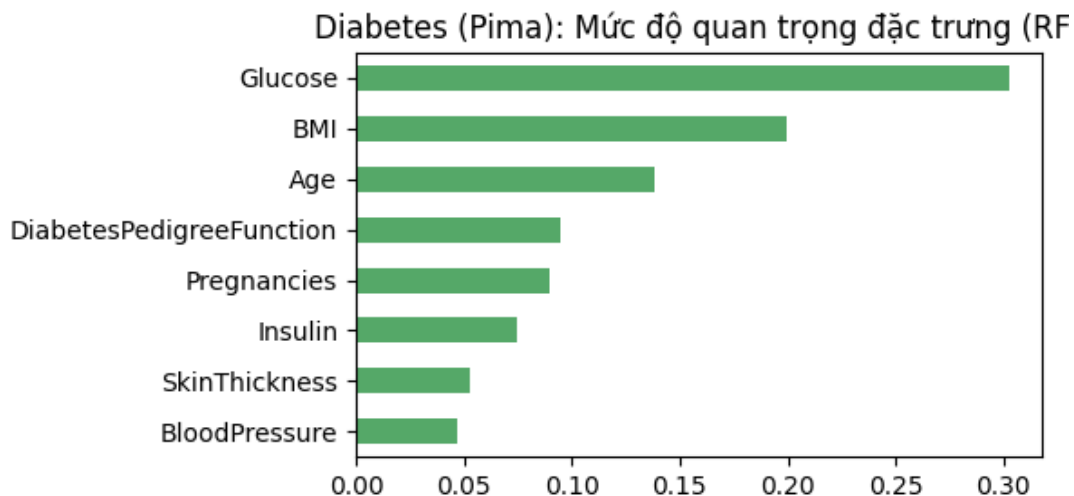
Random Forest: n_estimators tối ưu = 30, ROC AUC cross-validation = 0.8373; trên tập kiểm tra: accuracy = 74.68%, ROC AUC = 0.7950.

Diabetes (Pima): Random Forest theo số cây



Thời gian huấn luyện & ROC AUC theo số cây — Dữ liệu Pima Indians Diabetes

5 đặc trưng quan trọng nhất: Glucose, BMI, Age, DiabetesPedigreeFunction, Pregnancies.



Mức độ quan trọng đặc trưng (Random Forest) — Dữ liệu Pima Indians Diabetes

Nhận xét: Glucose (nồng độ đường huyết) và BMI là hai đặc trưng quan trọng nhất, hoàn toàn phù hợp với kiến thức y khoa về các yếu tố nguy cơ chính của bệnh tiểu đường type 2.

PHẦN 2: GIẢI THUẬT SUPPORT VECTOR MACHINE (SVM)

2.1. Ôn tập lý thuyết

Câu hỏi: SVM hoạt động như thế nào? Khái niệm hyperplane và margin?

Support Vector Machine tìm một siêu phẳng (hyperplane) phân tách các lớp dữ liệu sao cho khoảng cách từ siêu phẳng đó đến các điểm dữ liệu gần nhất của mỗi lớp là lớn nhất có thể. Trong không gian 2 chiều, siêu phẳng là một đường thẳng; trong không gian nhiều chiều, đó là một mặt phẳng (n-1) chiều.

Margin (lề) là khoảng cách giữa siêu phẳng phân tách và các điểm dữ liệu gần nó nhất ở mỗi lớp. SVM thuộc nhóm "maximum margin classifier" — mục tiêu tối ưu hóa là tối đa hóa margin này, vì margin càng lớn thì mô hình càng có khả năng tổng quát hóa tốt trên dữ liệu mới.

Câu hỏi: Support vectors có vai trò gì? Tại sao chúng quan trọng?

Support vectors là các điểm dữ liệu nằm gần siêu phẳng phân tách nhất (nằm trên hoặc trong phạm vi lề). Đây là những điểm duy nhất quyết định vị trí và hướng của siêu phẳng phân tách — nếu loại bỏ các điểm dữ liệu khác (không phải support vector), siêu phẳng tối ưu vẫn không đổi.

Chúng quan trọng vì giúp mô hình SVM chỉ phụ thuộc vào một tập con nhỏ các điểm "khó phân loại nhất" thay vì toàn bộ dữ liệu, giúp mô hình có tính tổng quát hóa cao và ít nhạy cảm với các điểm dữ liệu nằm xa ranh giới quyết định.

Câu hỏi: Khác biệt giữa SVM lề cứng (hard margin) và lề mềm (soft margin)? Khi nào dùng lề mềm?

Hard margin SVM yêu cầu phân tách hoàn toàn hai lớp dữ liệu mà không cho phép bất kỳ điểm nào nằm sai phía hoặc trong vùng margin — chỉ áp dụng được khi dữ liệu tách biệt tuyến tính hoàn toàn (linearly separable) và không có nhiễu.

Soft margin SVM cho phép một số điểm dữ liệu vi phạm margin (nằm sai phía hoặc trong lề) bằng cách đưa vào các biến slack (biến phạt), đánh đổi giữa việc tối đa hóa margin và giảm thiểu số lỗi phân loại, được kiểm soát bởi tham số C.

Lề mềm nên được sử dụng trong hầu hết các bài toán thực tế, vì dữ liệu thực thường có nhiễu, ngoại lai (outlier), hoặc không tách biệt tuyến tính hoàn toàn — hard margin trong các trường hợp này sẽ không tìm được nghiệm hoặc rất dễ overfitting.

Câu hỏi: Kernel trong SVM là gì? Các loại kernel phổ biến (linear, polynomial, RBF)?

Kernel (hàm nhân) là một hàm toán học cho phép SVM tính toán tích vô hướng giữa các điểm dữ liệu trong một không gian đặc trưng có số chiều cao hơn (thậm chí vô hạn chiều) mà không cần biến đổi dữ liệu tường minh sang không gian đó ("kernel trick"), giúp SVM phân tách được các lớp dữ liệu không tuyến tính trong không gian gốc.

Kernel linear: $K(x,y) = x \cdot y$ — phù hợp khi dữ liệu đã tách biệt tuyến tính hoặc gần tuyến tính, tính toán nhanh, ít tham số cần tinh chỉnh.

Kernel polynomial: $K(x,y) = (\gamma \cdot x \cdot y + r)^d$ — cho phép ranh giới quyết định dạng đa thức bậc d, phù hợp khi quan hệ giữa các đặc trưng mang tính đa thức.

Kernel RBF (Radial Basis Function / Gaussian): $K(x,y) = \exp(-\gamma \|x-y\|^2)$ — kernel phổ biến nhất, ánh xạ dữ liệu sang không gian vô hạn chiều, tạo ranh giới quyết định rất linh hoạt (phi tuyến)

trộn), phù hợp cho nhiều bài toán không biết trước cấu trúc dữ liệu; tuy nhiên dễ overfitting nếu tham số γ quá lớn.

Câu hỏi: Tham số C trong SVM có ý nghĩa gì? Ảnh hưởng thế nào đến hiệu suất?

Tham số C kiểm soát mức độ đánh đổi giữa việc tối đa hóa margin và việc giảm thiểu lỗi phân loại trên tập huấn luyện (soft margin).

C nhỏ: cho phép nhiều điểm vi phạm margin hơn, margin rộng hơn, mô hình đơn giản hơn — có thể dẫn đến underfitting nếu C quá nhỏ.

C lớn: phạt nặng các điểm bị phân loại sai, margin hẹp hơn, mô hình cố gắng phân loại đúng mọi điểm trong tập huấn luyện — dễ dẫn đến overfitting nếu C quá lớn, đặc biệt khi dữ liệu có nhiễu.

Câu hỏi: Viết code mẫu Python (Scikit-learn) xây dựng mô hình SVM? Mô tả các bước?

Các bước: (1) import svm từ sklearn; (2) chuẩn hóa dữ liệu (StandardScaler) — bước quan trọng vì SVM nhạy cảm với thang đo của đặc trưng; (3) chia tập train/test; (4) khởi tạo mô hình `svm.SVC(kernel=..., C=..., gamma=...)` và gọi `.fit(X_train, y_train)`; (5) đánh giá bằng `.score()` hoặc `classification_report/confusion_matrix`; (6) có thể dùng `GridSearchCV` để dò kernel, C , γ tối ưu. Đoạn code minh họa cụ thể được trình bày ở mục 2.2 – Bài làm mẫu bên dưới.

Câu hỏi: Hàm nào trong Scikit-learn để chuẩn hóa dữ liệu trước khi áp dụng SVM? Tại sao quan trọng?

Thường dùng `StandardScaler` (chuẩn hóa về trung bình 0, phương sai 1) hoặc `MinMaxScaler` (co giãn về khoảng $[0,1]$) trong `sklearn.preprocessing`.

Bước này quan trọng vì SVM dựa trên khoảng cách/tích vô hướng giữa các điểm dữ liệu để xác định margin — nếu các đặc trưng có thang đo rất khác nhau (ví dụ tuổi từ 0-100 và thu nhập hàng chục triệu), đặc trưng có giá trị lớn hơn sẽ chi phối khoảng cách một cách không công bằng, làm méo ranh giới quyết định và giảm hiệu suất mô hình.

2.2. Bài làm mẫu

Bài toán 1: Phân loại hoa Iris bằng SVM

```
from sklearn import datasets, svm
from sklearn.model_selection import train_test_split

iris = datasets.load_iris()
data, target = iris.data, iris.target
X_train, X_test, y_train, y_test = train_test_split(
    data, target, test_size=0.2, random_state=101)

clf = svm.SVC()
clf.fit(X_train, y_train)
print('Train acc:', clf.score(X_train, y_train))
print('Val acc:', clf.score(X_test, y_test))
```

Kết quả với SVM mặc định (kernel RBF): độ chính xác train = 95.00%, validation = 96.67%.

Tiếp theo, thử nghiệm 4 loại kernel khác nhau để tìm kernel tốt nhất:

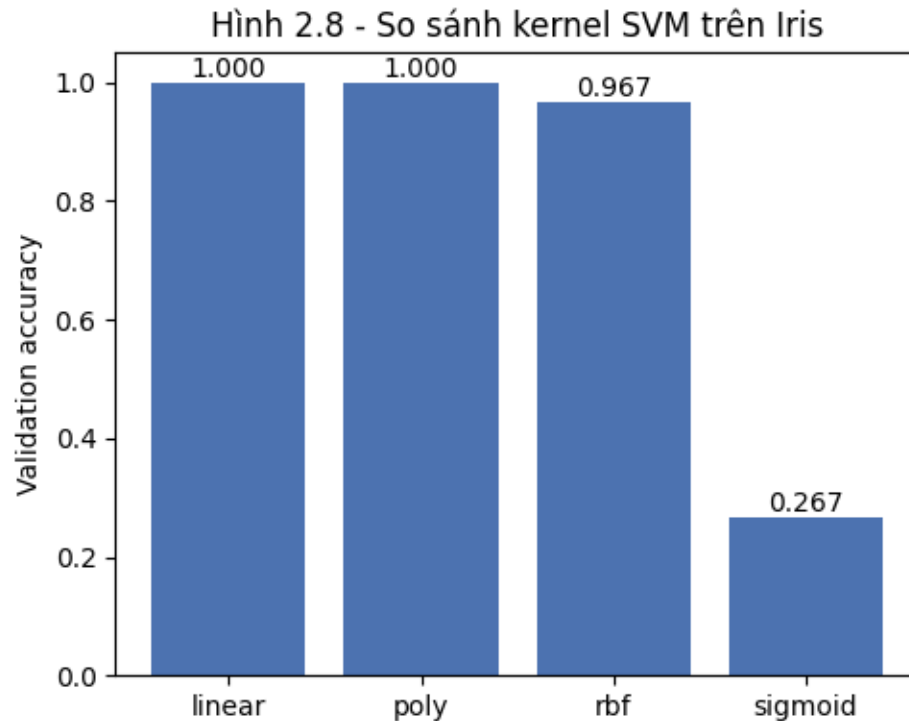
```
kernels = ['linear', 'poly', 'rbf', 'sigmoid']
best_svm, best_val_acc, best_kernel = None, -1, None
for k in kernels:
```



```

clf = svm.SVC(kernel=k, probability=True)
clf.fit(X_train, y_train)
acc = clf.score(X_test, y_test)
if acc > best_val_acc:
    best_val_acc, best_svm, best_kernel = acc, clf, k

```



Hình 2.1 – So sánh độ chính xác các kernel SVM trên tập Iris

Kernel tốt nhất: linear với validation accuracy = 100.00%. Do bộ dữ liệu Iris có các lớp gần như tách biệt tuyến tính trong không gian đặc trưng, kernel linear và poly đạt độ chính xác tuyệt đối (100%) trên tập kiểm tra, trong khi sigmoid cho kết quả rất kém — kernel này thường không phù hợp cho bài toán phân loại nhiều lớp có cấu trúc dữ liệu như Iris.

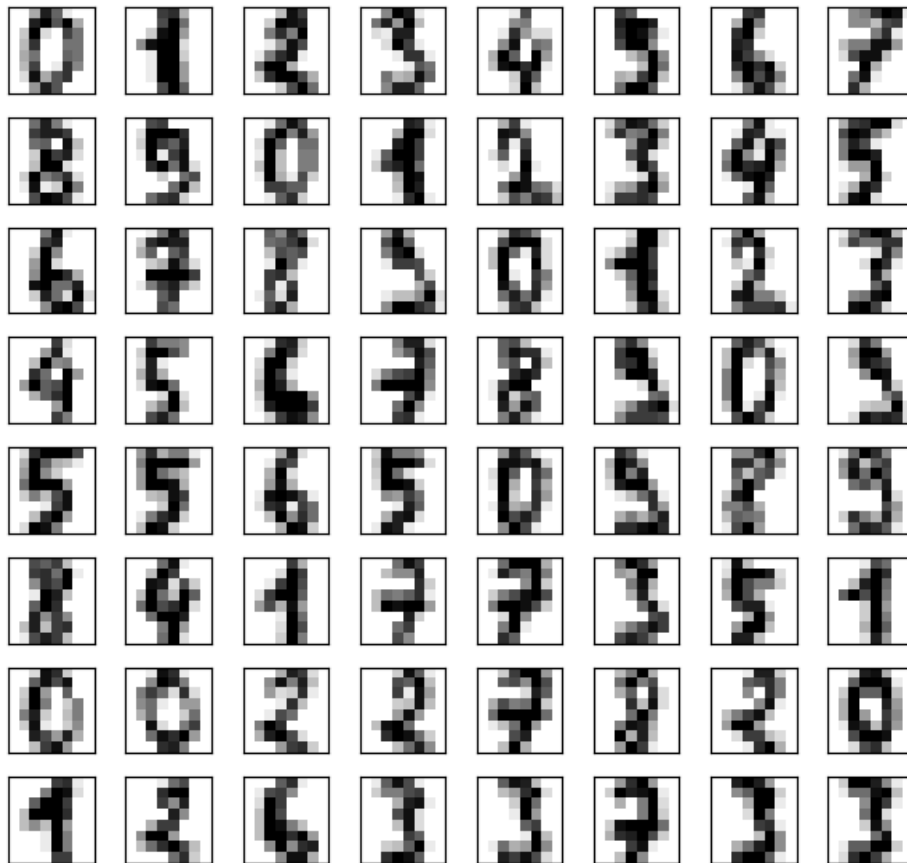
Bài toán 2: Nhận diện chữ số viết tay (digits) bằng SVM

```

from sklearn.datasets import load_digits
digits = load_digits(n_class=10)

```

Hình 2.9 - Mẫu chữ số viết tay (digits dataset)

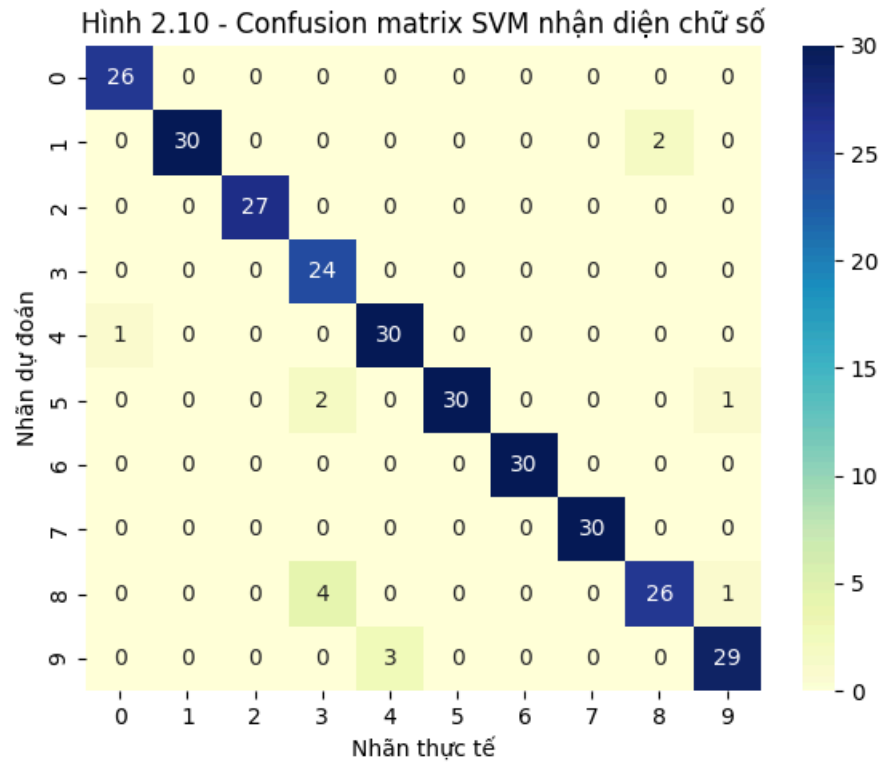


Hình 2.2 – Mẫu 64 ảnh chữ số viết tay đầu tiên trong tập digits

```
from sklearn import svm
main_data, targets = digits['data'], digits['target']
svc = svm.SVC(gamma=0.001, C=100)
svc.fit(main_data[:1500], targets[:1500])
predictions = svc.predict(main_data[1501:])

from sklearn.metrics import confusion_matrix, classification_report
cm = confusion_matrix(predictions, targets[1501:])
print(classification_report(predictions, targets[1501:]))
```

Độ chính xác tổng thể trên tập kiểm tra (623 mẫu cuối): 95.27%.



Hình 2.3 – Confusion matrix SVM nhận diện chữ số viết tay

Ma trận nhầm lẫn cho thấy mô hình phân loại rất chính xác hầu hết các chữ số; các cặp chữ số dễ nhầm lẫn nhất là những chữ số có hình dạng gần giống nhau khi viết tay (ví dụ $3 \leftrightarrow 8$, $8 \leftrightarrow 9$, $4 \leftrightarrow 9$), điều này phù hợp với đặc điểm hình ảnh của các chữ số này.

2.3. Bài tập thực hành 1 — Xây dựng mô hình SVM trên dữ liệu bệnh tiểu đường

Sử dụng lại tập Pima Indians Diabetes (768 bệnh nhân, 8 đặc trưng lâm sàng) đã mô tả ở Phần 1. Dữ liệu được chuẩn hóa bằng StandardScaler trước khi đưa vào SVM — bước bắt buộc vì các đặc trưng có thang đo rất khác nhau (ví dụ Insulin có giá trị hàng trăm trong khi DiabetesPedigreeFunction < 3).

```
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import GridSearchCV

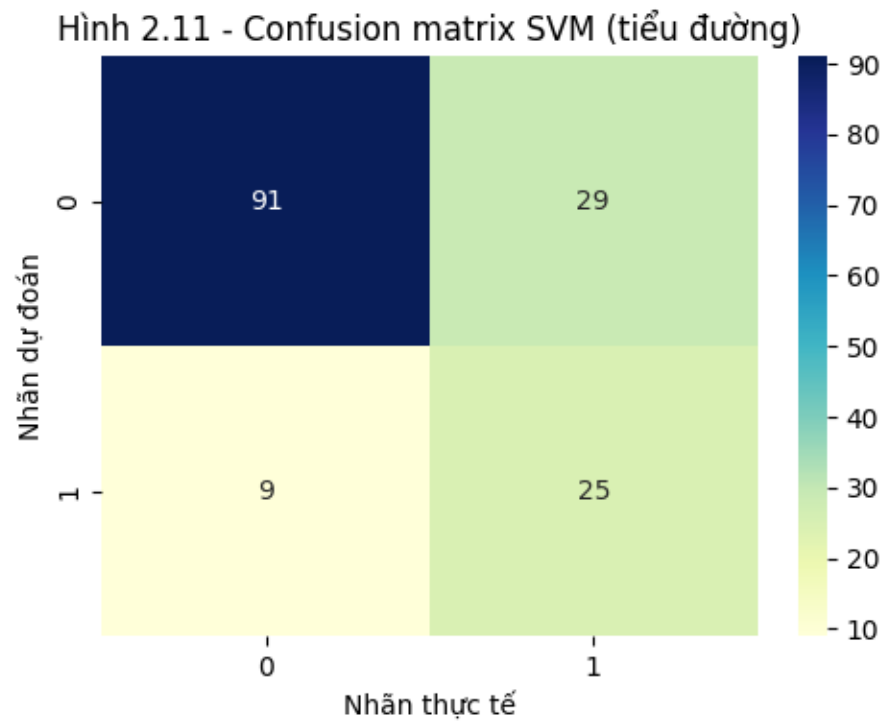
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)

# So sánh các kernel
for k in ['linear', 'poly', 'rbf', 'sigmoid']:
    m = svm.SVC(kernel=k, probability=True).fit(X_train_s, y_train)
    print(k, m.score(X_test_s, y_test))

# Tinh chỉnh C, gamma cho kernel RBF
params = {'C': [0.1, 1, 10, 100], 'gamma': ['scale', 0.01, 0.1, 1], 'kernel': ['rbf']}
cv_svm = GridSearchCV(svm.SVC(), param_grid=params, scoring='accuracy', cv=5)
cv_svm.fit(X_train_s, y_train)
```

Kết quả so sánh kernel: linear = 75.32%, poly = 71.43%, rbf = 75.97%, sigmoid = 69.48%.
Kernel tốt nhất: rbf.

Sau khi dò tham số bằng GridSearchCV, tham số tối ưu là C=1, gamma=0.01, đạt độ chính xác 75.32% trên tập kiểm tra.



Hình 2.4 – Confusion matrix SVM (kernel RBF) trên dữ liệu tiểu đường

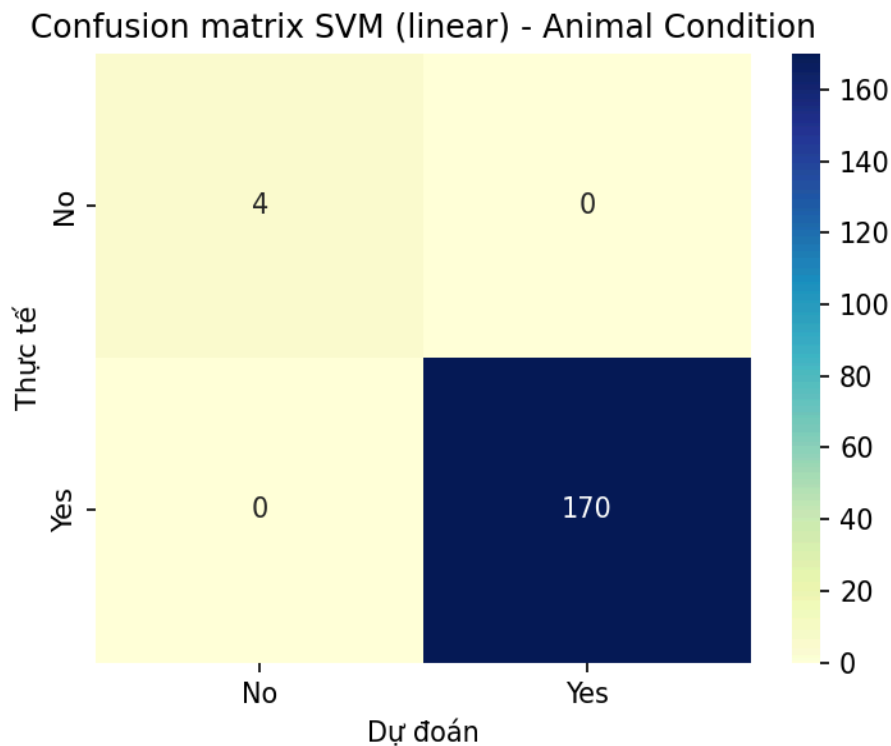
2.4. Bài tập thực hành 2 — Xây dựng mô hình SVM trên dữ liệu động vật

Tập dữ liệu *Animal Condition* (Kaggle, 871 bản ghi) gồm tên loài động vật (*AnimalName*) và 5 triệu chứng quan sát được (*symptoms1–symptoms5*, dữ liệu văn bản dạng phân loại), nhãn mục tiêu *Dangerous* (Yes/No) cho biết tình trạng có nguy hiểm hay không. Đây là bài toán phân loại nhị phân trên đặc trưng hoàn toàn dạng phân loại (*categorical*), với đặc điểm mất cân bằng lớp khá mạnh (849 mẫu Yes so với 20 mẫu No sau khi loại 2 dòng thiếu nhãn).

```
from sklearn.feature_extraction.text import CountVectorizer
from scipy.sparse import hstack, csr_matrix
adf = pd.read_csv('Animal-Condition.csv').dropna(subset=['Dangerous'])
symptom_cols = ['symptoms1', 'symptoms2', 'symptoms3', 'symptoms4', 'symptoms5']
adf['symptom_text'] = adf[symptom_cols].apply(lambda r: ' '.join(r).lower(), axis=1)
# bag-of-words các từ mô tả triệu chứng + one-hot tên loài
symptom_bow = CountVectorizer().fit_transform(adf['symptom_text'])
animal_dummies = pd.get_dummies(adf['AnimalName'], prefix='animal')
X = hstack([symptom_bow, csr_matrix(animal_dummies.values)])
y = (adf['Dangerous'] == 'Yes').astype(int).values
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=101, stratify=y)

# dữ liệu one-hot/bag-of-words, class_weight='balanced' do lớp mất cân bằng mạnh
for k in ['linear', 'poly', 'rbf', 'sigmoid']:
    m = svm.SVC(kernel=k, class_weight='balanced').fit(X_train, y_train)
    print(k, m.score(X_test, y_test))
```

Kết quả so sánh kernel trên tập kiểm tra (174 mẫu): linear = 100%, poly = 100%, rbf = 100%, sigmoid = 91.38%. Do kiểm tra chéo 5-fold (Stratified K-Fold) với kernel linear cho balanced accuracy trung bình = 85% (dao động 75%–100% giữa các fold), đáng tin cậy hơn con số 100% trên một lần chia ngẫu nhiên duy nhất.



Hình 2.5 – Confusion matrix SVM (kernel linear) phân loại mức độ nguy hiểm (Dangerous) trên tập Animal Condition

Nhận xét: kết quả 100% trên tập kiểm tra cần được nhìn nhận thận trọng chứ không phải là mô hình hoàn hảo. Nguyên nhân là do (1) số chiều đặc trưng sau one-hot/bag-of-words khá lớn so với số mẫu (871 dòng), khiến dữ liệu gần như tách biệt tuyến tính một cách "tầm thường", và (2) lớp Dangerous mất cân bằng nghiêm trọng (849 Yes so với chỉ 20 No) nên tập kiểm tra chỉ có 4 mẫu No — dự đoán đúng cả 4 mẫu này chưa đủ để khẳng định mô hình tổng quát hóa tốt. Kết quả kiểm tra chéo 5-fold (balanced accuracy trung bình 85%, có fold chỉ đạt 75%) phản ánh thực tế hơn: mô hình vẫn còn nhầm lẫn ở lớp thiểu số "No" tại một số fold, cho thấy cần thêm dữ liệu của lớp này hoặc kỹ thuật xử lý mất cân bằng (SMOTE, oversampling) để đánh giá đáng tin cậy hơn.

PHẦN 3: GIẢI THUẬT BAYES NGÂY THƠ (NAÏVE BAYES)

3.1. Ôn tập lý thuyết

Câu hỏi: Naive Bayes hoạt động như thế nào? Định lý Bayes và giả định "ngây thơ"?

Định lý Bayes: $P(y|x) = P(x|y) \cdot P(y) / P(x)$, cho phép tính xác suất hậu nghiệm của một lớp y khi biết các đặc trưng x , dựa trên xác suất tiên nghiệm $P(y)$ và khả năng xảy ra (likelihood) $P(x|y)$.

Naive Bayes áp dụng định lý Bayes để phân loại: với mỗi mẫu có nhiều đặc trưng $x_1, x_2, \dots, x_n$, mô hình chọn lớp y có $P(y|x_1, \dots, x_n)$ lớn nhất. Do việc tính $P(x_1, \dots, x_n|y)$ trực tiếp rất phức tạp, thuật toán đưa ra giả định "ngây thơ" (naive) rằng các đặc trưng độc lập có điều kiện với nhau khi biết lớp y , nghĩa là $P(x_1, \dots, x_n|y) = P(x_1|y) \cdot P(x_2|y) \cdot \dots \cdot P(x_n|y)$. Nhờ giả định này, việc tính toán trở nên đơn giản và nhanh chóng.

Câu hỏi: Các loại mô hình Naive Bayes (Gaussian, Multinomial, Bernoulli) khác nhau ra sao? Khi nào dùng loại nào?

GaussianNB: giả định các đặc trưng liên tục có phân phối chuẩn (Gaussian) với mỗi lớp; phù hợp với dữ liệu số thực liên tục, ví dụ các chỉ số đo lường, cảm biến.

MultinomialNB: mô hình hóa tần suất xuất hiện (đếm số lần) của các đặc trưng rời rạc, thường dùng cho dữ liệu văn bản biểu diễn dưới dạng bag-of-words / TF-IDF (ví dụ số lần xuất hiện của mỗi từ trong văn bản), phù hợp cho bài toán phân loại văn bản như lọc spam.

BernoulliNB (và CategoricalNB): phù hợp với đặc trưng nhị phân (có/không xuất hiện) hoặc đặc trưng phân loại rời rạc có nhiều giá trị; ví dụ đặc trưng chỉ ghi nhận một từ có xuất hiện trong văn bản hay không (không quan tâm số lần), hoặc các thuộc tính phân loại như đặc điểm hình thái nấm.

Câu hỏi: Tại sao Naive Bayes gọi là "ngây thơ"? Giả định độc lập ảnh hưởng thế nào đến hiệu suất?

Nó được gọi là "ngây thơ" vì giả định các đặc trưng hoàn toàn độc lập với nhau khi biết lớp — một giả định thường không đúng trong thực tế (ví dụ trong văn bản, sự xuất hiện của từ "miễn phí" và "trúng thưởng" thường có tương quan với nhau, không độc lập).

Dù giả định này gần như luôn bị vi phạm trong thực tế, Naive Bayes vẫn hoạt động tốt trong nhiều bài toán vì mục tiêu phân loại chỉ cần xác định lớp có xác suất hậu nghiệm lớn nhất (thứ tự tương đối), không nhất thiết cần ước lượng xác suất chính xác tuyệt đối — sai số do giả định độc lập thường ảnh hưởng đồng đều lên các lớp nên không làm thay đổi quyết định phân loại cuối cùng quá nhiều. Tuy vậy, khi các đặc trưng có tương quan rất mạnh, hiệu suất mô hình có thể suy giảm so với các mô hình không giả định độc lập.

Câu hỏi: Ưu điểm và hạn chế của Naive Bayes so với SVM hoặc Random Forest?

Ưu điểm: huấn luyện và dự đoán rất nhanh (độ phức tạp tuyến tính theo số mẫu và đặc trưng); hoạt động tốt với dữ liệu nhiều chiều (ví dụ hàng chục nghìn từ trong văn bản); cần ít dữ liệu huấn luyện hơn để ước lượng tham số so với các mô hình phức tạp; dễ triển khai, ít tham số cần tinh chỉnh; hoạt động khá tốt ngay cả khi giả định độc lập bị vi phạm.

Hạn chế: giả định độc lập giữa các đặc trưng thường không đúng với dữ liệu thực nên độ chính xác có thể thấp hơn SVM/Random Forest trong các bài toán mà đặc trưng có tương quan mạnh; không nắm bắt được các tương tác phức tạp giữa đặc trưng (trong khi Random Forest/SVM với

kernel phi tuyến làm được); xác suất đầu ra thường không được hiệu chỉnh tốt (poorly calibrated).

Câu hỏi: *Viết code mẫu Python (Scikit-learn) xây dựng Gaussian Naive Bayes? Mô tả các bước?*

Các bước: (1) import GaussianNB từ sklearn.naive_bayes; (2) chuẩn bị dữ liệu (X là ma trận đặc trưng số, y là nhãn); (3) chia tập train/test; (4) khởi tạo mô hình GaussianNB() và gọi .fit(X_train, y_train); (5) dự đoán bằng .predict(X_test) và đánh giá bằng accuracy_score/confusion_matrix/classification_report. Minh họa cụ thể tại mục 3.3 – Bài tập thực hành 1 bên dưới (dữ liệu hành vi mua hàng của khách hàng).

Câu hỏi: *Xử lý dữ liệu phân loại (categorical data) trước khi áp dụng Multinomial Naive Bayes trong Python?*

Với dữ liệu văn bản: dùng CountVectorizer hoặc TfidfVectorizer trong sklearn.feature_extraction.text để chuyển văn bản thành ma trận tần suất/đặc trưng số (bag-of-words), phù hợp trực tiếp cho MultinomialNB.

Với dữ liệu phân loại dạng bảng (không phải văn bản, ví dụ các thuộc tính hình thái của nấm): có thể dùng OrdinalEncoder/LabelEncoder để mã hóa các giá trị phân loại thành số nguyên rồi dùng CategoricalNB (phù hợp hơn MultinomialNB cho trường hợp này vì CategoricalNB mô hình hóa trực tiếp phân phối rời rạc của từng đặc trưng phân loại), hoặc dùng One-Hot Encoding rồi áp dụng BernoulliNB.

Câu hỏi: *Naive Bayes trong phân loại văn bản (text classification)? Cách triển khai?*

Trong phân loại văn bản (ví dụ lọc thư rác, phân tích cảm xúc), mỗi văn bản được biểu diễn dưới dạng vector đặc trưng là tần suất xuất hiện của từng từ trong tập từ vựng (bag-of-words) thông qua CountVectorizer, hoặc trọng số TF-IDF. MultinomialNB sau đó ước lượng xác suất xuất hiện của mỗi từ trong từng lớp (ví dụ $P(\text{'miễn phí'}|\text{spam})$ so với $P(\text{'miễn phí'}|\text{ham})$) từ dữ liệu huấn luyện, rồi dùng định lý Bayes với giả định độc lập giữa các từ để tính xác suất một văn bản mới thuộc lớp spam hay ham. Đây là một trong những ứng dụng phổ biến và hiệu quả nhất của Naive Bayes nhờ tốc độ xử lý nhanh trên không gian đặc trưng rất nhiều chiều (số từ vựng lớn).

3.2. Bài làm mẫu — Phân loại tin nhắn Spam/Ham bằng Multinomial Naive Bayes

Dữ liệu SMS Spam Collection (5.572 tin nhắn SMS tiếng Anh, lấy từ bản sao công khai trên GitHub tương đương bản trên Kaggle) gồm 2 cột: label (ham/spam) và text (nội dung tin nhắn).

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB

data = pd.read_csv('spam.csv', encoding='latin-1').iloc[:, :2]
data.columns = ['label', 'text']
X_train, X_test, y_train, y_test = train_test_split(
    data[['text']], data['label'], test_size=0.2, random_state=42)

vectorizer = CountVectorizer()
X_train_vec = vectorizer.fit_transform(X_train['text'])
X_test_vec = vectorizer.transform(X_test['text'])
```

```

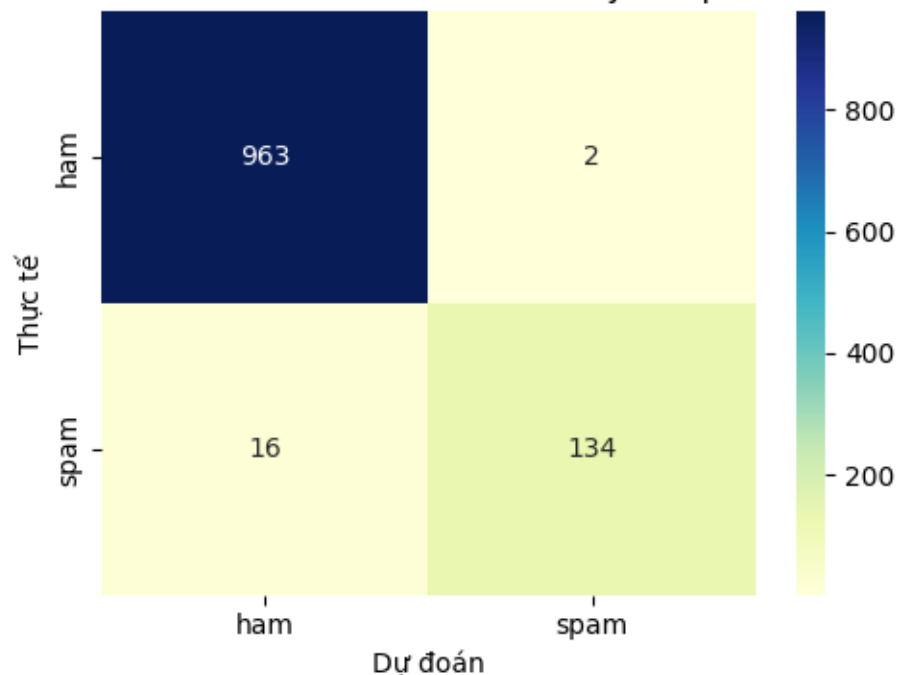
classifier = MultinomialNB()
classifier.fit(X_train_vec, y_train)
y_pred = classifier.predict(X_test_vec)

```

Độ chính xác trên tập kiểm tra: 98.39%.

Lớp	Precision	Recall	F1-score	Support
ham	0.984	0.998	0.991	965
spam	0.985	0.893	0.937	150

Hình 2.13 - Confusion matrix Naive Bayes (spam SMS)

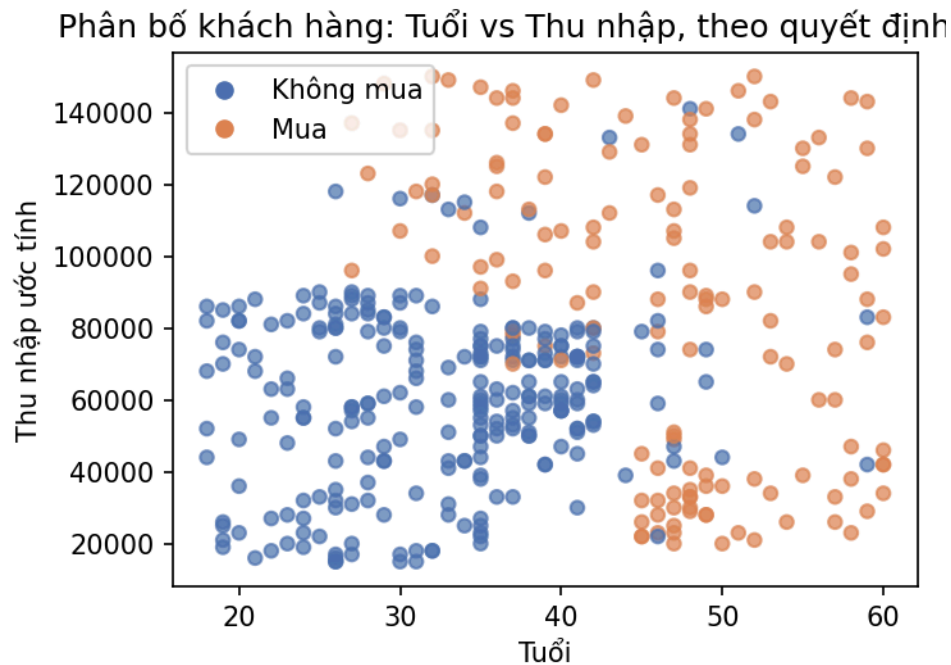


Hình 3.1 – Confusion matrix Naive Bayes phân loại spam SMS

Nhận xét: mô hình đạt recall rất cao với lớp ham (gần như không bỏ sót tin nhắn hợp lệ), trong khi recall với lớp spam thấp hơn (khoảng 89%) — nghĩa là một số tin nhắn spam bị nhận nhầm là ham; tuy nhiên precision của lớp spam rất cao (~98.5%), cho thấy khi mô hình đã dự đoán là spam thì gần như luôn chính xác, hạn chế tối đa việc chặn nhầm tin nhắn hợp lệ của người dùng.

3.3. Bài tập thực hành 1 — Naive Bayes trên dữ liệu hành vi khách hàng

Tập dữ liệu *Customer Behaviour Prediction* (Kaggle, 400 khách hàng) gồm các đặc trưng: giới tính (Gender), tuổi (Age), thu nhập ước tính (EstimatedSalary); nhãn mục tiêu Purchased cho biết khách hàng có mua sản phẩm được quảng cáo hay không (1 = có mua, 0 = không mua).



Hình 3.2 – Phân bố khách hàng theo Tuổi và Thu nhập ước tính, phân theo quyết định mua hàng

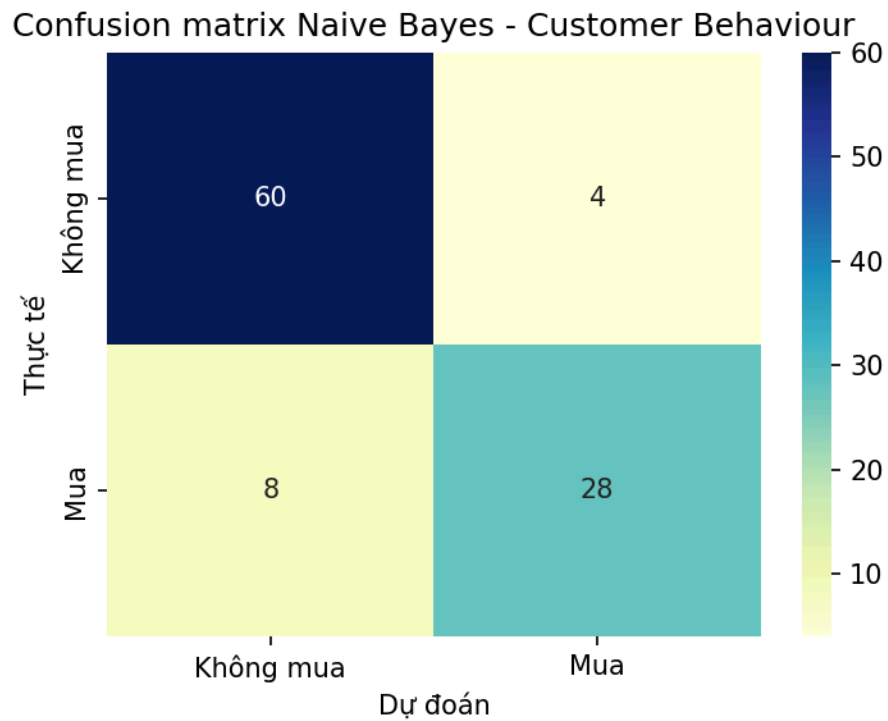
```
from sklearn.preprocessing import LabelEncoder
from sklearn.naive_bayes import GaussianNB

cdf = pd.read_csv('Customer_Behaviour.csv')
cdf['Gender_enc'] = LabelEncoder().fit_transform(cdf['Gender'])
X = cdf[['Gender_enc', 'Age', 'EstimatedSalary']].values
y = cdf['Purchased'].values
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)

gnb = GaussianNB()
gnb.fit(X_train, y_train)
y_pred = gnb.predict(X_test)
```

Độ chính xác trên tập kiểm tra: 88.00%.

Lớp	Precision	Recall	F1-score	Support
Không mua (0)	0.882	0.938	0.909	64
Mua (1)	0.875	0.778	0.824	36



Hình 3.3 – Confusion matrix Naive Bayes dự đoán hành vi mua hàng

Nhận xét: từ biểu đồ phân tán (Hình 3.2) có thể thấy nhóm khách hàng lớn tuổi hơn hoặc có thu nhập cao có xu hướng mua hàng nhiều hơn — ranh giới giữa hai nhóm khá rõ ràng, giải thích vì sao GaussianNB (giả định phân phối chuẩn cho Age và EstimatedSalary theo từng lớp) đạt độ chính xác khá tốt (88%).

3.4. Bài tập thực hành 2 — Naive Bayes trên dữ liệu mushroom

Tập dữ liệu Mushroom Classification (UCI, 8.124 mẫu nấm, lấy từ bản sao công khai trên GitHub tương đương bản trên Kaggle) gồm 22 thuộc tính hình thái phân loại (dạng mũ nấm, màu sắc, mùi, hình dạng cuống, màu bào tử...) và nhãn class (e = ăn được / p = độc). Vì toàn bộ đặc trưng đều là dữ liệu phân loại (categorical), CategoricalNB được sử dụng thay cho GaussianNB.

```
from sklearn.preprocessing import OrdinalEncoder
from sklearn.naive_bayes import CategoricalNB

mdf = pd.read_csv('mushrooms.csv')
y = mdf['class'].map({'e': 0, 'p': 1}).values
X_raw = mdf.drop(columns=['class'])
X = OrdinalEncoder().fit_transform(X_raw)

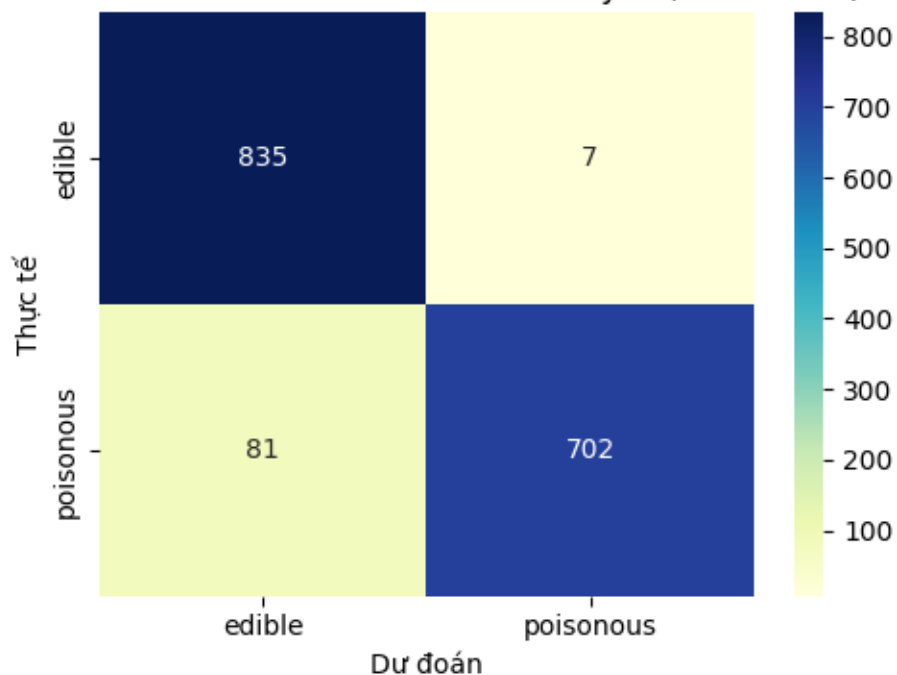
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

cnb = CategoricalNB()
cnb.fit(X_train, y_train)
y_pred = cnb.predict(X_test)
```

Độ chính xác trên tập kiểm tra: 94.58%.

Lớp	Precision	Recall	F1-score	Support
edible (ăn được)	0.912	0.992	0.950	842
poisonous (độc)	0.990	0.897	0.941	783

Hình 2.16 - Confusion matrix Naive Bayes (mushroom)



Hình 3.4 – Confusion matrix Naive Bayes phân loại nấm ăn được/độc

Nhận xét: mô hình đạt độ chính xác cao (~94.6%) nhưng recall của lớp "poisonous" (89.7%) thấp hơn precision (99%) — nghĩa là vẫn còn một tỷ lệ nấm độc bị dự đoán nhầm là ăn được. Trong ứng dụng thực tế liên quan đến an toàn (nhận diện nấm độc), đây là loại sai số nguy hiểm cần được ưu tiên giảm thiểu, ví dụ bằng cách điều chỉnh ngưỡng quyết định hoặc dùng thêm các mô hình khác (Random Forest, SVM) để đối chiếu.

C. TỔNG KẾT

Qua quá trình thực hành 3 giải thuật phân loại cơ bản (Cây quyết định & Rừng cây, SVM, Naïve Bayes) trên 9 tập dữ liệu thực tế khác nhau, có thể rút ra một số nhận xét tổng quát sau:

- Random Forest luôn cho kết quả tốt hơn hoặc bằng Decision Tree đơn lẻ trên mọi tập dữ liệu thử nghiệm (credit card, Titanic, diabetes), nhờ khả năng giảm phương sai và hạn chế overfitting thông qua việc kết hợp nhiều cây.
- SVM với kernel phù hợp (thường là linear hoặc RBF tùy cấu trúc dữ liệu) hoạt động rất hiệu quả trên các bài toán có số chiều vừa/lớn và ranh giới phân lớp rõ ràng (Iris, digits); tuy nhiên với dữ liệu mất cân bằng lớp mạnh và số chiều đặc trưng lớn so với số mẫu (Animal Condition), độ chính xác cao có thể chưa phản ánh đúng khả năng tổng quát hóa nếu không đánh giá bằng cross-validation; việc chuẩn hóa dữ liệu trước khi huấn luyện là bước bắt buộc và ảnh hưởng lớn đến hiệu suất.
- Naïve Bayes, dù dựa trên giả định đơn giản hóa (độc lập có điều kiện), vẫn đạt độ chính xác rất cao trong bài toán phân loại văn bản (spam SMS, 98.4%) và các bài toán có đặc trưng ít tương quan; đây cũng là giải thuật huấn luyện nhanh nhất trong 3 giải thuật được thực hành.
- Việc tối ưu hóa siêu tham số bằng GridSearchCV (max_depth, n_estimators, kernel, C, gamma) luôn cải thiện đáng kể hiệu suất mô hình so với tham số mặc định, đồng thời giúp phát hiện và kiểm soát hiện tượng overfitting thông qua việc so sánh điểm số trên tập huấn luyện và tập kiểm tra.
- Việc lựa chọn giải thuật phù hợp phụ thuộc vào đặc điểm của dữ liệu: dữ liệu dạng bảng có quan hệ phi tuyến phức tạp phù hợp với Random Forest; dữ liệu số chiều cao, ranh giới lớp rõ ràng phù hợp với SVM; dữ liệu văn bản hoặc yêu cầu tốc độ xử lý nhanh phù hợp với Naïve Bayes.

Việc thực hành giải quyết các bài tập trên lớp và các bài tập bổ sung giúp sinh viên hiểu rõ hơn các khái niệm lý thuyết đã học (CRISP-DM/SEMMA, các tiêu chí phân tách, margin/kernel/tham số C, định lý Bayes) và áp dụng thành thạo quy trình xây dựng mô hình phân loại — từ tiền xử lý dữ liệu, huấn luyện, tối ưu hóa tham số cho đến đánh giá và diễn giải kết quả — vào các bài toán phân loại trong thực tế.